FFT convolution uses the overlap-add method shown in Fig. 18-1; only the way that the input segments are converted into the output segments is changed. Figure 18-2 shows an example of how an input segment is converted into an output segment by FFT convolution. To start, the frequency response of the filter is found by taking the DFT of the filter kernel, using the FFT. For instance, (a) shows an example filter kernel, a windowed-sinc band-pass filter. The FFT converts this into the real and imaginary parts of the frequency response, shown in (b) & (c). These frequency domain signals may not *look* like a band-pass filter because they are in rectangular form. Remember, polar form is usually best for humans to understand the frequency domain, while rectangular form is normally best for mathematical calculations. These real and imaginary parts are stored in the computer for use when each segment is being calculated.

Figure (d) shows the input segment to being processed. The FFT is used to find its frequency spectrum, shown in (e) & (f). The frequency spectrum of the output segment, (h) & (i) is then found by multiplying the filter's frequency response, (b) & (c), by the spectrum of the input segment, (e) & (f). Since these spectra consist of real and imaginary parts, they are multiplied according to Eq. 9-1 in Chapter 9. The Inverse FFT is then used to find the output segment, (g), from its frequency spectrum, (h) & (i). It is important to recognize that this output segment is exactly the same as would be obtained by the direct convolution of the input segment, (d), and the filter kernel, (a).

The FFTs must be long enough that *circular convolution* does not take place (also described in Chapter 9). This means that the FFT should be the same length as the output segment, (g). For instance, in the example of Fig. 18-2, the filter kernel contains 129 points and each segment contains 128 points, making output segment 256 points long. This calls for 256 point FFTs to be used. This means that the filter kernel, (a), must be padded with 127 zeros to bring it to a total length of 256 points. Likewise, each of the input segments, (d), must be padded with 128 zeros. As another example, imagine you need to convolve a very long signal with a filter kernel having 600 samples. One alternative would be to use segments of 425 points, and 1024 point FFTs. Another alternative would be to use segments of 1449 points, and 2048 point FFTs.

Table 18-1 shows an example program to carry out FFT convolution. This program filters a 10 million point signal by convolving it with a 400 point filter kernel. This is done by breaking the input signal into 16000 segments, with each segment having 625 points. When each of these segments is convolved with the filter kernel, an output segment of $625 + 400 - 1 = 1024$ points is produced. Thus, 1024 point FFTs are used. After defining and initializing all the arrays (lines 130 to 230), the first step is to calculate and store the frequency response of the filter (lines 250 to 310). Line 260 calls a mythical subroutine that loads the filter kernel into XX[0] through XX[399], and sets XX[400] through XX[1023] to a value of zero. The subroutine in line 270 is the FFT, transforming the 1024 samples held in XX[ ] into the 513 samples held in REX[ ] & IMX[ ], the real and